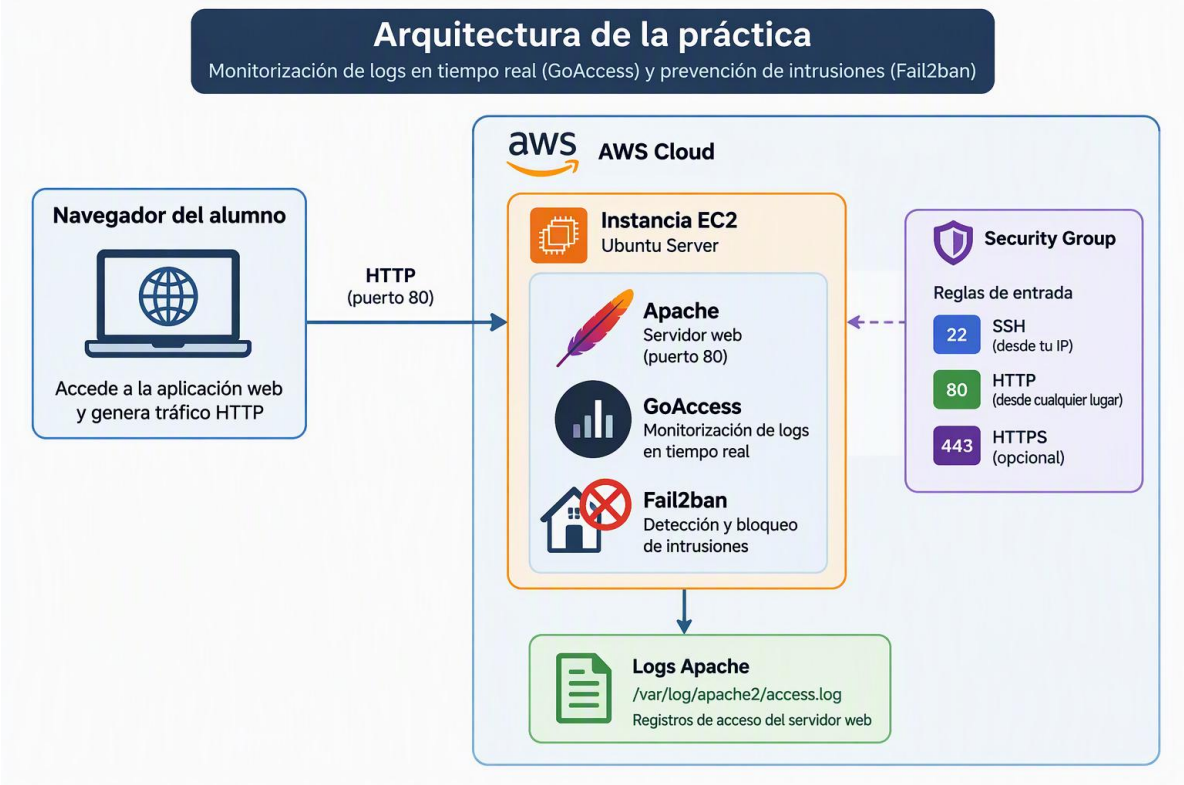
Práctica 4: Monitorización de Logs en Tiempo Real (GoAccess) y Prevención de Intrusiones (Fail2ban) en AWS Academy Learner Lab

1. Objetivos

- Desplegar una instancia EC2 en AWS.
- Configurar un servidor web Apache.
- Monitorizar accesos en tiempo real mediante GoAccess.
- Proteger recursos mediante autenticación básica.
- Detectar y bloquear actividades sospechosas con Fail2ban.
- Analizar registros de seguridad.
- Elaborar un informe técnico de evidencias.

---

2. Arquitectura de la práctica



---

3. Creación de la instancia EC2

3.1 Acceder a AWS Academy Learner Lab

1. Entrar en AWS Academy.
2. Iniciar el laboratorio.
3. Abrir AWS Console.

3.2 Crear instancia EC2

Seleccionar:

Nombre: Ut2-nombreAlumno-practica-goaccess-fail2ban

AMI: Ubuntu Server 24.04 LTS

Tipo de instancia: t3.micro

Claves: vockey

Descargar el fichero .pem.

3.3 Security Group

Crear un nuevo grupo de seguridad.

Reglas de entrada

Tipo	Puerto	Origen
SSH	22	Mi IP
HTTP	80	Anywhere
HTTPS	443	Anywhere

Si vemos que da problemas podemos poner SSH Anywhere, pero en entornos productivos habría que poner la IP del administrador.

---

4. User Data de Inicialización

Copiar el siguiente script completo en la sección:

Advanced Details → User Data

Shell

```
#!/bin/bash
apt update -y
apt install apache2 -y
apt install goaccess -y
apt install fail2ban -y
apt install apache2-utils -y
systemctl enable apache2
systemctl start apache2

echo "<h1>Servidor Despliegue AWS del alumno XXXX</h1>" >
/var/www/html/index.html

touch /var/log/apache2/access.log

systemctl enable fail2ban

cat > /etc/fail2ban/jail.local << EOF
[DEFAULT]
bantime = 600
findtime = 300
maxretry = 10

[apache-404]
enabled = true
filter = apache-404
logpath = /var/log/apache2/access.log
backend = auto
maxretry = 10
findtime = 300
bantime = 600
EOF

cat > /etc/fail2ban/filter.d/apache-404.conf << EOF
[Definition]
failregex = ^<HOST> - .* "(GET|POST|HEAD).*" 404
ignoreregex =
EOF

systemctl restart fail2ban

echo "Instalacion finalizada correctamente" > /home/ubuntu/instalacion.txt
```

Pulsamos lanzar instancia

5. Conexión SSH

Shell
chmod 400 practica-despliegue.pem

ssh -i practica-despliegue.pem ubuntu@IP_PUBLICA

Comprobar:

Shell
Hostnamectl

```
ubuntu@ip-172-31-47-154:~$ hostnamectl
 Static hostname: ip-172-31-47-154
       Icon name: computer-vm
         Chassis: vm
      Machine ID: ec253ea328e79a8aa9547d532e140cd0
         Boot ID: 41543d4fec9248f6b2d723e2e378dd41
  Virtualization: amazon
Operating System: Ubuntu 24.04.4 LTS
          Kernel: Linux 6.17.0-1017-aws
    Architecture: x86-64
 Hardware Vendor: Amazon EC2
  Hardware Model: t3.micro
Firmware Version: 1.0
   Firmware Date: Mon 2017-10-16
    Firmware Age: 8y 11month 3w 6d
ubuntu@ip-172-31-47-154:~$
```

6. Verificación inicial

Comprobar Apache:

Shell
sudo systemctl status apache2

```
ubuntu@ip-172-31-47-154:~$ sudo systemctl status apache2
● apache2.service - The Apache HTTP Server
     Loaded: loaded (/usr/lib/systemd/system/apache2.service; enabled; preset: enabled)
     Active: active (running) since Tue 2026-10-13 10:30:42 UTC; 7min ago
       Docs: https://httpd.apache.org/docs/2.4/
   Main PID: 1910 (apache2)
      Tasks: 55 (limit: 1015)
     Memory: 5.4M (peak: 5.7M)
        CPU: 67ms
     CGroup: /system.slice/apache2.service
             ├─1910 /usr/sbin/apache2 -k start
             ├─1912 /usr/sbin/apache2 -k start
             └─1913 /usr/sbin/apache2 -k start
```

Comprobar navegador:

http://IP_PUBLICA

Debe aparecer:



---

7. Configuración de GoAccess

GoAccess es una herramienta de análisis de logs web en tiempo real que permite visualizar la actividad de un servidor mediante paneles interactivos accesibles desde el navegador.

En vez de revisar */var/log/apache2/access.log* podemos ver estadísticas fáciles de interpretar

``

7.1 Generar informe HTML

Ejecutar:

```
sudo goaccess \
/var/log/apache2/access.log \
--log-format=COMBINED \
-o /var/www/html/monitor.html
```

7.2 Crear usuario de acceso

Shell
sudo htpasswd -c /etc/apache2/.htpasswd alumno

Introducir contraseña (cada uno la que quiera)

7.3 Proteger monitor.html

Crear archivo:

Shell
sudo nano /etc/apache2/conf-available/monitor.conf

Contenido:

```
<Location "/monitor.html">
AuthType Basic
AuthName "Monitorizacion"
AuthUserFile /etc/apache2/.htpasswd
Require valid-user
</Location>
```

Activar:

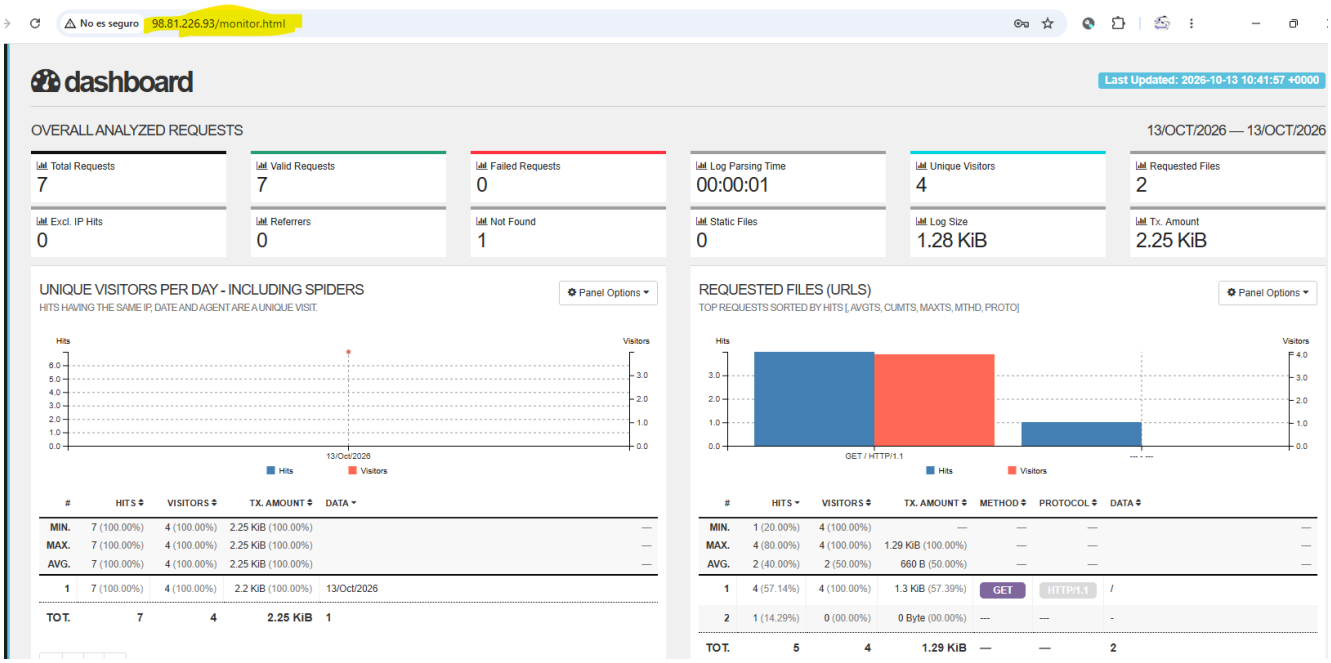
```
sudo a2enconf monitor

sudo systemctl restart apache2
```

7.4 Probar acceso

Abrir: *http://IP_PUBLICA/monitor.html*

Debe solicitar usuario y contraseña.



8. Configuración de GoAccess en tiempo real

Crear:

Shell
sudo nano /etc/systemd/system/goaccess.service

```
[Unit]
Description=GoAccess Real Time

[Service]
ExecStart=/usr/bin/goaccess \
/var/log/apache2/access.log \
--log-format=COMBINED \
-o /var/www/html/monitor.html \
--real-time-html

Restart=always

[Install]
WantedBy=multi-user.target
```

Guardamos el fichero y lo activamos

Activar:

```
Shell
sudo systemctl daemon-reload

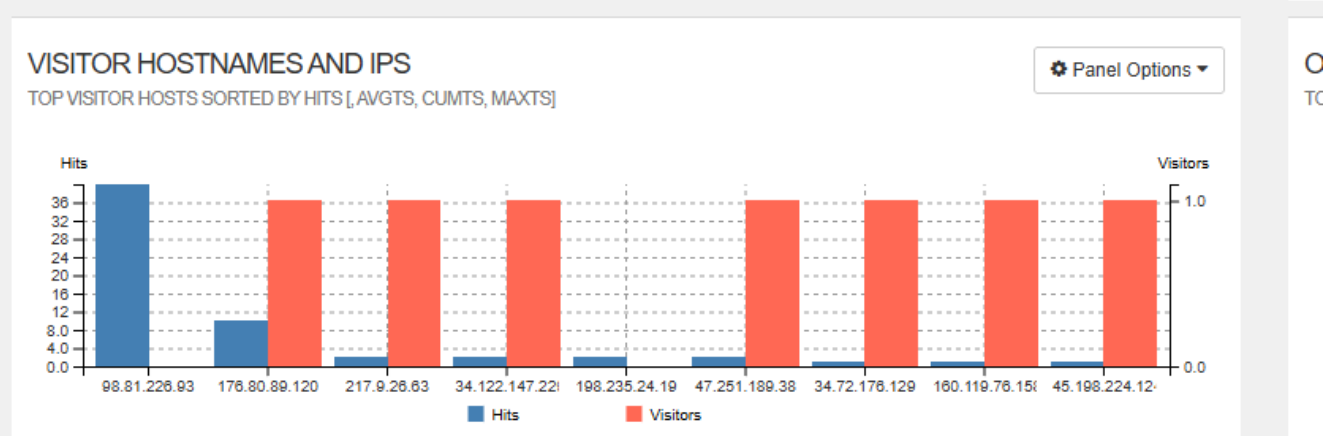
sudo systemctl enable goaccess

sudo systemctl start goaccess
```

Verificar:

```
Shell
sudo systemctl status goaccess
```

Si vemos la página de monitorización, vemos por ejemplo :



La primera es nuestra IP, con varias peticiones porque he lanzado un

for i in {1..20}; do curl http://98.81.226.93/noexiste$i; done

Pero vemos también estas IPs:

- 98.81.226.93
- 176.80.89.120
- 217.9.26.63
- 34.122.147.221
- 198.235.24.19

Se corresponden a

- Bots automáticos de Internet.
- Escáneres de vulnerabilidades.
- Motores de búsqueda.
- Herramientas que recorren Internet buscando servidores expuestos.

Para ver qué han intentado hacer cogemos una IP por ejemplo y vemos el access.log

sudo grep "176.80.89.120" /var/log/apache2/access.log

```
ubuntu@ip-172-31-47-154:~$ sudo grep "176.80.89.120" /var/log/apache2/access.log
``
176.80.89.120 - - [13/Oct/2026:10:40:35 +0000] "GET / HTTP/1.1" 200 330 "-" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (K
HTML, like Gecko) Chrome/155.0.0.0 Safari/537.36"
176.80.89.120 - - [13/Oct/2026:10:40:35 +0000] "GET /favicon.ico HTTP/1.1" 404 490 "http://98.81.226.93/" "Mozilla/5.0 (Windows NT 10.0; Win
64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/155.0.0.0 Safari/537.36"
176.80.89.120 - - [13/Oct/2026:10:41:27 +0000] "-" 408 0 "-" "-"
176.80.89.120 - - [13/Oct/2026:11:48:43 +0000] "GET /monitor.html HTTP/1.1" 401 727 "-" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebK
it/537.36 (KHTML, like Gecko) Chrome/155.0.0.0 Safari/537.36"
176.80.89.120 - alumno [13/Oct/2026:11:48:48 +0000] "GET /monitor.html HTTP/1.1" 200 157717 "-" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) A
ppleWebKit/537.36 (KHTML, like Gecko) Chrome/155.0.0.0 Safari/537.36"
176.80.89.120 - - [13/Oct/2026:11:49:40 +0000] "-" 408 0 "-" "-"
176.80.89.120 - alumno [13/Oct/2026:11:50:52 +0000] "GET /monitor.html HTTP/1.1" 200 157717 "-" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) A
ppleWebKit/537.36 (KHTML, like Gecko) Chrome/155.0.0.0 Safari/537.36"
176.80.89.120 - alumno [13/Oct/2026:11:50:54 +0000] "GET /monitor.html HTTP/1.1" 200 157716 "-" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) A
ppleWebKit/537.36 (KHTML, like Gecko) Chrome/155.0.0.0 Safari/537.36"
176.80.89.120 - alumno [13/Oct/2026:11:51:09 +0000] "GET /monitor.html HTTP/1.1" 200 157717 "-" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) A
```

9. Configuración de Fail2ban

Fail2ban es una herramienta de seguridad para Linux que analiza los registros (logs) de los servicios y bloquea automáticamente las direcciones IP que muestran comportamientos sospechosos.

Verificar: *sudo systemctl status fail2ban*

Estado: *sudo fail2ban-client status*

Salida esperada:

```
ubuntu@ip-172-31-47-154:~$ sudo fail2ban-client status
Status
|- Number of jail:      2
`- Jail list:   apache-404, sshd
ubuntu@ip-172-31-47-154:~$
```

10. Simulación de ataque

Desde el ordenador del alumno:

Linux (cada uno que ponga **la IP de su servidor EC2**) :

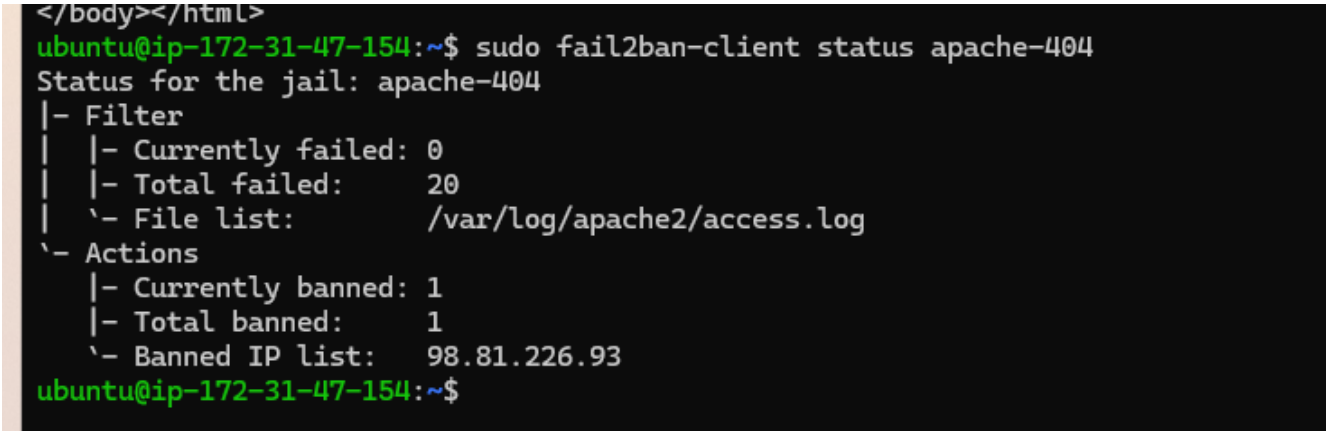
for i in {1..20}; do curl http://98.81.226.93/noexiste$i; done

11. Verificación del bloqueo

Comprobar:

Shell
sudo fail2ban-client status apache-404

Deberá aparecer: Banned IP list con la IP atacante.



12. Actividad de Ampliación (por parejas)

Blue Team vs Red Team: Protección de un servidor web frente a ataques distribuidos

Una empresa ha desplegado un servidor web en AWS protegido mediante **Fail2ban** y monitorizado con **GoAccess**. Sin embargo, los administradores sospechan que algunos atacantes están utilizando múltiples direcciones IP para evitar los mecanismos de bloqueo tradicionales.

El objetivo de esta actividad es comprender tanto las capacidades como las limitaciones de Fail2ban, así como la importancia de la monitorización y el análisis de logs.

Organización

Cada pareja estará formada por:

- **Red Team (Atacante)**
- **Blue Team (Defensor)**

Objetivos

Red Team

Simular comportamientos maliciosos contra el servidor web intentando evitar los mecanismos de detección basados en IP.

Ataque 1: Generar peticiones hacia múltiples URLs inexistentes:

- Generar errores 404.
- Comprobar si Fail2ban detecta el comportamiento.

Ataque 2: Simulación de ataque distribuido:

- repartir las peticiones entre varios miembros de la clase o varios dispositivos disponibles (ordenador, portátil, móvil conectado a otra red, etc.), de forma que cada origen genere pocas peticiones.

Ataque 3: Tráfico mixto. Combinar:

- Peticiones válidas.
- Peticiones erróneas.
- Accesos rápidos consecutivos.

Blue Team

Detectar los ataques utilizando GoAccess y los registros del sistema, mejorar la configuración de Fail2ban y proponer medidas adicionales de mitigación.

Analizar:

- Hosts más activos.
- URLs más solicitadas.
- Códigos de respuesta.
- Volumen de tráfico generado.

Registrar:

- Número de IPs bloqueadas.
- Intentos detectados.
- Limitaciones observadas.

Infraestructura

- instancia AWX-EC2 Ubuntu
- Apache o Nginx
- GoAccess
- Fail2ban
- Acceso SSH

Medidas adicionales

Además de Fail2ban, proponer al menos tres medidas para mejorar la protección.

---

13. Cuestiones finales

Pregunta 1

¿Qué diferencia existe entre monitorización y prevención de intrusiones?

Pregunta 2

¿Qué ventajas aporta GoAccess respecto a revisar directamente los logs?

Pregunta 3

¿Qué es un ataque de fuerza bruta?

Pregunta 4

¿Qué es una respuesta HTTP 404?

Pregunta 5

¿Qué ocurriría si el valor de maxretry se estableciera en 1?

Pregunta 6

¿Qué otros servicios además de Apache podrían protegerse mediante Fail2ban?